Facultad de Ingeniería en
Ciencias de la Computación y Telecomunicaciones
U.A.G.R.M.

Somos ingeniería!

# UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO

## FACULTAD DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN Y TELECOMUNICACIONES

**SEGUNDO PARCIAL**

**“DRESSED — Plataforma SaaS multitenant para tiendas de ropa con Vestidor Virtual Inteligente”**

Proyecto de Desarrollo de Software con ICONIX y UML

“Grupo 19”

Docente: MSc. Ing. Angélica Garzón Cuéllar

Materia: Sistemas de Información 2

Semestre: 2 - 2026

Integrantes: Carrasco Surco Cinthia — 221025121

Fecha: ____ /____ /2026

# TABLA DE CONTENIDO

CAPÍTULO I. ASPECTOS GENERALES.....4
1.1. Introducción.....4
1.2. Planteamiento del problema.....4
1.3. Objetivos del proyecto.....5
1.3.1. Objetivo general.....5
1.3.2. Objetivos específicos.....5
1.4. Justificación.....5
1.5. Alcance y limitaciones.....6
1.6. Metodología de desarrollo ICONIX.....6
CAPÍTULO II. ANÁLISIS DE REQUISITOS.....7
2.1. Descripción del sistema propuesto.....7
2.2. Identificación de actores.....7
2.3. Requisitos funcionales y no funcionales.....8
Requisitos funcionales.....8
Requisitos no funcionales.....10
2.4. Reglas de negocio.....11
2.5. Diagrama UML de casos de uso.....12
2.6. Especificación de casos de uso.....12
CU-09 Administrar cajas de la sucursal.....12
CU-13 Abrir y cerrar sesión de caja.....12
CU-14 Registrar venta en POS (efectivo / tarjeta / QR).....13
CU-16 Sincronizar operaciones offline.....13
CU-21 Consultar catálogo público de la empresa.....13
CU-22 Registrarse como cliente de una tienda.....13

CU-10 Personalizar y compartir mi tienda web....13
CU-23 Probar prenda en el vestidor virtual....14
CU-19 Solicitar reporte con IA....14
CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX....14
3.1. Modelo de dominio....14
3.2. Diagramas de robustez....14
3.3. Diagramas de secuencia UML....15
3.4. Diagrama de clases UML....16
3.5. Diseño de la base de datos....16
3.6. Diseño de interfaces de usuario....17
CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN....17
4.1. Herramientas y tecnologías utilizadas....17
4.2. Diseño de la arquitectura del sistema....18
4.3. Implementación de la base de datos....19
4.4. Desarrollo de los módulos del sistema....19
4.5. Integración de componentes....21
CAPÍTULO V. PRUEBAS Y RESULTADOS....21
5.1. Plan de pruebas....21
5.2. Pruebas funcionales....22
5.3. Pruebas de integración....22
5.4. Corrección de errores....22
5.5. Resultados de las pruebas....23
5.6. Casos de prueba....23
CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES....25
6.1. Conclusiones....25




6.2. Recomendaciones....26

BIBLIOGRAFÍA....26

ANEXOS....27

Anexo A. Matriz de trazabilidad (requisito → caso de uso → implementación → prueba)....27

Anexo B. Diccionario de datos....29

Anexo C. Guía de usuario....41

Cuentas de demostración (contraseña 123456)....41

Cliente....41

Cajero....41

Encargado de sucursal....41

Administrador....42

Anexo D. Cómo reproducir....42

# CAPÍTULO I. ASPECTOS GENERALES

## 1.1. Introducción

DRESSED es una plataforma **SaaS multitenant** para negocios de venta de ropa. Una sola instalación del sistema atiende a varias empresas; cada una administra sus sucursales, personal, catálogo, inventario, cajas, reservas y ventas, y obtiene **su propia tienda en línea** accesible por un enlace compartible (`/t/<código-de-empresa>`). Los clientes finales compran y reservan desde la web o desde la aplicación móvil.

El sistema se compone de tres productos que comparten una única API:

| Producto | Tecnología | Usuarios |
|---|---|---|
| API | NestJS, TypeORM, PostgreSQL | Todos |
| Aplicación web (PWA) | React + Vite | Personal de la empresa y clientes |
| Aplicación móvil | Expo / React Native | Clientes |

## 1.2. Planteamiento del problema

Los comercios de ropa pequeños y medianos controlan inventario, ventas y reservas con hojas de cálculo o sistemas aislados; no pueden mostrar su catálogo en línea ni operar si se cae la conexión; no saben qué

prendas reponer ni quiénes son sus mejores clientes; y los clientes no pueden probar una prenda antes de reservarla. Las empresas con varias sucursales además pierden el control de quién abre cada caja, quién vende y quién modificó qué dato.

Un software hecho a la medida de una sola tienda no resuelve el problema del proveedor del software: necesita vender **el mismo sistema a muchas empresas** garantizando que los datos de una nunca sean visibles para otra.

## 1.3. Objetivos del proyecto

### 1.3.1. Objetivo general

Desarrollar una plataforma SaaS multitenant, modelada con ICONIX y UML, que permita a cada empresa de ropa gestionar su operación comercial y publicar su catálogo en línea para sus clientes, con aislamiento total de datos.

### 1.3.2. Objetivos específicos

1. Modelar el sistema con ICONIX (requisitos, casos de uso, robustez, secuencia y clases) manteniendo los modelos coherentes con el código (RNF12).
2. Implementar el aislamiento multitenant en tres capas: API, ORM y base de datos.
3. Ofrecer planes de suscripción que habiliten o limiten funciones (sucursales, IA, operación offline).
4. Implementar el flujo operativo de la tienda: sucursales, personal con roles, catálogo, inventario, cajas, sesiones de caja, POS y reservas.
5. Permitir operar sin conexión (ventas en efectivo y reservas) con sincronización idempotente y gestión de conflictos.
6. Dar a cada empresa una tienda web propia, personalizable y compartible, y una app móvil donde el cliente inicia sesión y ve solo el catálogo de su empresa.
7. Incorporar analítica: dashboard, reportes, reportes en lenguaje natural con IA y modelos de ML.
8. Registrar una bitácora de auditoría de las acciones del personal.

## 1.4. Justificación

- **Técnica:** una sola base de código y una sola base de datos sirven a todas las empresas, reduciendo el costo de operación.
- **Económica:** el plan Básico es gratuito; las empresas crecen de plan sin migrar datos.
- **Social/comercial:** las tiendas pequeñas acceden a herramientas de punto de venta, catálogo en línea y analítica que antes solo tenían las grandes.
- **Académica:** aplica ICONIX de punta a punta con trazabilidad requisito → caso de uso → clase → prueba.

## 1.5. Alcance y limitaciones

**Alcance.** Plataforma SaaS (alta de empresas y planes), panel de administración por rol, POS con cajas, reservas, tienda pública por empresa, vestidor virtual, PWA offline, app móvil de clientes, reportes, IA y ML, bitácora, datos demo.

**Limitaciones conocidas.**

- Un **correo electrónico pertenece a una sola empresa** (`users.email` y `clients.email` son únicos globalmente).
- Categorías, tallas, colores, temporadas, ciudades, colecciones, promociones y proveedores son catálogos globales compartidos.
- La app móvil es de clientes; las ventas offline se hacen en el POS web. El cliente móvil debe conocer el **código de tienda** al registrarse por primera vez.
- Sin `AI_API_KEY`, los reportes con IA usan un intérprete local en español (no lenguaje libre arbitrario).
- La personalización de la tienda web cubre portada, color, logo, contacto y secciones; no es un constructor de páginas libre ni admite dominio propio.
- El vestidor virtual depende de la cámara y de que la imagen de la prenda permita acceso desde el navegador (CORS).

## 1.6. Metodología de desarrollo ICONIX

ICONIX es un proceso **dirigido por casos de uso** y ligero, que usa un subconjunto de UML. Las etapas aplicadas:

| Etapa ICONIX | Producto en este proyecto | Capítulo |
|---|---|---|
| Requisitos | Requisitos funcionales / no funcionales, reglas de negocio | II |
| Modelo de dominio | `docs/uml/src/02_modelo_dominio.puml` | III |
| Casos de uso | `01_casos_de_uso.puml` y especificaciones | II |
| Diagramas de robustez | `03a…03e_robustez_*.puml` | III |
| Diagramas de secuencia | `04a…04g_sec_*.puml` | III |
| Diagrama de clases | generado desde las entidades + `05_clases_diseno.puml` | III |
| Implementación y pruebas | Código, pruebas Jest y E2E | IV, V |

Se utiliza **PlantUML** como lenguaje de modelado: los modelos son texto versionable en Git y se renderizan con un script (`docs/uml/scripts/build_uml.sh`).

# CAPÍTULO II. ANÁLISIS DE REQUISITOS

## 2.1. Descripción del sistema propuesto

DRESSED publica en la raíz (`/`) una **página comercial** que presenta el software, sus planes y el registro de empresas. Cada empresa registrada obtiene:

1. Un **panel de administración** (`/dashboard`) con menú según el rol del usuario.
2. Una **tienda en línea propia** en `/t/<código>` (por ejemplo `/t/luna`), compartible con sus clientes y personalizable por el administrador en *Mi tienda web*.
3. Una **aplicación móvil** donde el cliente debe iniciar sesión con su correo y ve el catálogo de la empresa a la que pertenece.

Operación típica: el administrador crea sucursales, personal y cajas; el encargado administra su sucursal; el cajero abre una sesión de caja y vende en el POS (aun sin conexión, en efectivo); el cliente consulta el catálogo, prueba la prenda en el vestidor virtual, la reserva o la compra y paga en línea o en sucursal.

## 2.2. Identificación de actores

| Actor | Tipo | Responsabilidad |
|---|---|---|
| SuperAdmin | Humano (plataforma) | Consulta y administra todas las empresas del SaaS. |
| Administrador | Humano (empresa) | Dueño: sucursales, personal, catálogo, plan, tienda web, reportes, auditoría. Administra cajas y sesiones desde su panel. |
| Encargado | Humano (sucursal) | Gestiona su sucursal: inventario, cajas, ventas, reservas, conflictos, reportes de su sucursal. |
| Cajero | Humano (sucursal) | Abre/cierra su sesión de caja, vende en POS, entrega reservas. |
| Cliente | Humano (externo) | Consulta el catálogo de **su** empresa, prueba, reserva, compra, ve su historial. |
| Stripe / pasarela | Sistema externo | Cobro en línea y webhook de confirmación. |
| Cloudinary | Sistema externo | Almacenamiento de imágenes de productos. |
| SMTP / Firebase | Sistema externo | Códigos por correo y notificaciones push. |
| LLM (opcional) | Sistema externo | Interpreta la pregunta del reporte |

| Actor | Tipo | Responsabilidad |
|---|---|---|
| | | en lenguaje natural. |

## 2.3. Requisitos funcionales y no funcionales

### Requisitos funcionales

| ID | Requisito | Módulo |
|---|---|---|
| RF-01 | Registrar una empresa con su administrador, elegir plan y verificar el correo con un código. | tenants |
| RF-02 | Mostrar planes y permitir al administrador cambiar de plan (con historial). | tenants |
| RF-03 | Iniciar sesión, cambiar contraseña y activar cuentas de clientes por código de correo. | auth |
| RF-04 | Administrar sucursales respetando el máximo del plan. | branches |
| RF-05 | Administrar personal con roles (Administrador, Encargado, Cajero); Encargado y Cajero pertenecen a una sucursal. | auth |
| RF-06 | Administrar catálogo: productos, variantes (talla/color), categorías, temporadas, colecciones, promociones. | catalog |
| RF-07 | Subir imágenes de producto (Cloudinary) y las partes para el vestidor virtual. | catalog |
| RF-08 | Gestionar inventario por sucursal, movimientos y proveedores. | inventory |
| RF-09 | **Crear cajas** por sucursal; luego abrir y cerrar sesiones de caja (un cajero, una caja abierta). | sales |
| RF-10 | Registrar ventas en POS (efectivo, tarjeta, QR) con cliente opcional. | sales |
| RF-11 | Registrar ventas en efectivo y | sync |

| ID | Requisito | Módulo |
|---|---|---|
| | reservas **sin conexión** y sincronizarlas sin duplicar. | |
| RF-12 | Detectar conflictos al sincronizar (stock, precio, caja cerrada, datos inválidos) y resolverlos. | sync |
| RF-13 | Publicar el catálogo de cada empresa en `/t/<código>`; sin código válido no se muestra nada. | store |
| RF-14 | Registro de clientes **dentro de una empresa**; en móvil el acceso al catálogo exige iniciar sesión. | auth, mobile |
| RF-15 | Reservar prendas en una sucursal y pagarlas en línea (Stripe) o por QR; entrega y cobro en sucursal. | reservations |
| RF-16 | Comprar en línea (checkout web) y consultar mis compras y reservas. | sales |
| RF-17 | Probar prendas con el vestidor virtual (cámara + detección de pose). | frontend |
| RF-18 | El administrador personaliza su tienda web (portada, logo, color, contacto, secciones, publicar) y la comparte por enlace. | tenants |
| RF-19 | Dashboard con indicadores y reportes de ventas, productos, inventario y reservas, con dictado por voz. | dashboard, reports |
| RF-20 | Reportes en lenguaje natural con IA (planes con IA). | ai-reports |
| RF-21 | Pronóstico de demanda y segmentación RFM de clientes (ML). | analytics |
| RF-22 | Bitácora de auditoría filtrable y exportable. | audit |
| RF-23 | Configuración de apariencia: | frontend |

| ID | Requisito | Módulo |
|---|---|---|
| | tema, idioma, moneda, sidebar compacto. | |
| RF-24 | Notificaciones al cliente (tiempo real y push). | notifications |
| RF-25 | Generar datos demostrativos (2 empresas, personal, cajas, ventas, bitácora). | scripts |

### Requisitos no funcionales

| ID | Requisito |
|---|---|
| RNF-01 | Aislamiento de datos en 3 capas: API (JWT + TenantGuard), ORM (`TenantSubscriber`) y BD (FKs compuestas). |
| RNF-02 | El `tenantId` nunca proviene del cliente en operaciones autenticadas: sale del JWT. En rutas públicas se resuelve por el slug de la tienda y no existe “empresa por defecto”. |
| RNF-03 | Sincronización idempotente (UUID por operación) y atómica (transacción + bloqueo pesimista). |
| RNF-04 | La IA nunca genera SQL: produce una especificación validada contra listas blancas; consulta parametrizada, de solo lectura y con tiempo límite. |
| RNF-05 | La aplicación web es una PWA instalable y navegable sin conexión. |
| RNF-06 | Los datos históricos se preservan (migración idempotente). |
| RNF-07 | Interfaz adaptable (responsive) a móvil, tableta y escritorio. |
| RNF-08 | Las contraseñas se almacenan con hash; las acciones del personal se auditan. |
| RNF-09 | El procesamiento de pose del vestidor virtual ocurre en el dispositivo del cliente (privacidad). |
| RNF-10 | Los datos demo pueden regenerarse sin tocar empresas reales (`seed:demo:reset`). |
| RNF-11 | Control de acceso por rol y por plan en cada endpoint (`RolesGuard`, `PlanFeatureGuard`). |

| ID | Requisito |
|---|---|
| **RNF-12** | **Coherencia documental: los modelos UML deben representar la solución realmente implementada.** Se cumple generando los diagramas de clases/BD desde las entidades y verificando los demás contra el código. |

## 2.4. Reglas de negocio

| ID | Regla |
|---|---|
| RN-01 | Toda entidad operativa pertenece a una sola empresa; ninguna consulta puede cruzar empresas. |
| RN-02 | Un correo electrónico solo puede pertenecer a una empresa. |
| RN-03 | El número de sucursales activas no puede superar el máximo del plan. |
| RN-04 | Plan Básico ($0): 1 sucursal, sin reportes. Starter ($29): 3 sucursales y reportes básicos. Profesional ($79): 10 sucursales, reportes con IA y analítica ML. Empresarial ($199): ilimitadas. |
| RN-05 | Encargado y Cajero deben estar asignados a una sucursal. |
| RN-06 | El administrador **no entra a una sucursal como espacio de trabajo aparte**: administra todo desde su propio panel lateral; el encargado tiene un panel equivalente, adaptado y limitado a su sucursal. |
| RN-07 | Primero se **crea** una caja (única por sucursal y nombre); después un cajero **abre una sesión** sobre ella. |
| RN-08 | Un cajero no puede tener dos cajas abiertas y una caja no puede estar abierta dos veces a la vez. |
| RN-09 | Sin conexión solo se aceptan ventas en **efectivo**; tarjeta y QR requieren conexión. |
| RN-10 | Una operación offline se aplica una sola vez (idempotencia por UUID). |
| RN-11 | Si lo vendido offline supera el stock real, se aplica y se registra un conflicto pendiente para el encargado. |
| RN-12 | Un cliente solo ve el catálogo de la empresa a la que pertenece; en móvil debe iniciar sesión. |

| ID | Regla |
|---|---|
| RN-13 | Una tienda despublicada muestra «no disponible» a los clientes. |
| RN-14 | El reporte con IA solo está disponible en planes con `aiGenerativeReports`. |
| RN-15 | Las contraseñas de las cuentas demo son `123456`; los usuarios nuevos deben cambiar su contraseña temporal. |
| RN-16 | Toda acción de alta, cambio o baja del personal queda en la bitácora con usuario, IP y fecha. |

## 2.5. Diagrama UML de casos de uso

| Diagrama de casos de uso |
|---|

*Diagrama de casos de uso*

Fuente: `docs/uml/src/01_casos_de_uso.puml`. Casos de uso: CU-01 Registrar empresa · CU-02 Gestionar plan · CU-03 Iniciar sesión · CU-04 Landing y planes · CU-05 Sucursales · CU-06 Personal y roles · CU-07 Catálogo · CU-08 Inventario · CU-09 Cajas · CU-10 Tienda web · CU-11 Bitácora · CU-12 Apariencia · CU-13 Sesión de caja · CU-14 Venta POS · CU-15 Entregar reserva · CU-16 Sincronizar · CU-17 Conflictos · CU-18 Dashboard/reportes · CU-19 Reporte IA · CU-20 Analítica ML · CU-21 Catálogo público · CU-22 Registro de cliente · CU-23 Vestidor · CU-24 Reservar · CU-25 Comprar · CU-26 Mis reservas/compras · CU-27 Notificaciones.

## 2.6. Especificación de casos de uso

### CU-09 Administrar cajas de la sucursal

**Actores:** Administrador, Encargado. **Precondición:** la sucursal existe.

*Curso básico:* 1) El actor abre «Sesiones de caja» y pulsa **Crear caja**. 2) Indica sucursal (el encargado solo la suya) y nombre. 3) El sistema valida que el nombre sea único en la sucursal y crea la caja activa. 4) La caja aparece como disponible para abrir.

*Alternos:* 3a) Nombre repetido → se rechaza. 4a) Se desactiva una caja → ya no se ofrece al abrir turno.

### CU-13 Abrir y cerrar sesión de caja

**Actor:** Cajero. **Precondición:** su sucursal tiene al menos una caja activa y libre.

*Curso básico:* 1) El cajero abre su caja con un monto inicial; si hay una sola libre, el sistema la elige. 2) Vende (CU-14). 3) Al terminar cierra e informa el efectivo contado. 4) El sistema calcula esperado vs. reportado y registra la diferencia.

*Alternos:* 1a) Ya tiene una caja abierta → rechazo. 1b) La caja está en uso → rechazo. 1c) No hay cajas creadas → mensaje para pedir a la administración que cree una.

### CU-14 Registrar venta en POS (efectivo / tarjeta / QR)

**Actor:** Cajero. **Precondición:** caja abierta.

*Curso básico:* 1) Busca productos y los agrega al carrito. 2) Opcionalmente asocia o crea un cliente. 3) Elige método de pago y cobra. 4) El sistema registra venta, ítems y pago, descuenta inventario y asocia la venta a la sesión de caja. 5) Muestra el comprobante.

*Alternos:* 3a) Sin conexión y efectivo → se guarda en la cola local (CU-16). 3b) Sin conexión y tarjeta/QR → se informa que solo se acepta efectivo. 4a) Stock insuficiente → se rechaza la línea.

### CU-16 Sincronizar operaciones offline

**Actor:** sistema (sin intervención). **Disparador:** vuelve la conexión.

*Curso básico:* 1) El gestor de sincronización lee las operaciones pendientes. 2) Las envía en lote. 3) El servidor las ordena y, por cada una, verifica si su UUID ya fue aplicado; si es nueva, la ejecuta en una transacción con bloqueo de inventario. 4) Devuelve el resultado por operación. 5) El cliente elimina las aplicadas. *Alternos:* 3a) Detecta stock/precio/caja cerrada → aplica con conflicto pendiente (CU-17).

### CU-21 Consultar catálogo público de la empresa

**Actor:** Cliente (web). **Precondición:** conoce el enlace o el código de la tienda.

*Curso básico:* 1) Abre `/t/<código>`. 2) El sistema valida la empresa y la tienda publicada. 3) Muestra portada personalizada, productos destacados y sucursales. 4) El cliente navega al catálogo y al detalle de cada producto. *Alternos:* 2a) Código inexistente → «Tienda no encontrada». 2b) Tienda despublicada → «no disponible». 2c) Sesión de otra empresa → aviso para cerrar sesión o volver a su tienda.

### CU-22 Registrarse como cliente de una tienda

**Actor:** Cliente. **Precondición:** conoce el código de su tienda.

*Curso básico:* 1) Abre `/register?tenant=<código>` (en móvil escribe el código de tienda). 2) Completa sus datos. 3) El sistema crea usuario y cliente **dentro de esa empresa**. 4) Inicia sesión y es llevado a su tienda. *Alternos:* 1a) Sin código → se le pide. 3a) Correo ya existente sin cuenta digital → se le ofrece activar su cuenta por código. 3b) Código inexistente → error 404.

### CU-10 Personalizar y compartir mi tienda web

**Actor:** Administrador.

*Curso básico:* 1) Abre «Mi tienda web». 2) Edita frase, título, descripción, logo (URL https), color de acento, WhatsApp, Instagram y qué secciones mostrar. 3) Guarda; el sistema sanea los valores. 4) Revisa

la vista previa. 5) Copia o comparte el enlace por WhatsApp/menú del dispositivo. *Alternos:* 2a) Desactiva «Tienda publicada» → el enlace muestra «no disponible».

### CU-23 Probar prenda en el vestidor virtual

**Actor:** Cliente. **Precondición:** el producto tiene realidad aumentada habilitada.

*Curso básico:* 1) Abre el vestidor desde el detalle. 2) Autoriza la cámara. 3) El sistema detecta la pose en el propio dispositivo. 4) Recorta el espacio transparente de la prenda y la ajusta a hombros/caderas/rodillas. *Alternos:* 2a) Permiso denegado → mensaje. 3a) Cuerpo no detectado → se pide alejarse.

### CU-19 Solicitar reporte con IA

**Actor:** Administrador / Encargado. **Precondición:** el plan incluye IA.

*Curso básico:* 1) Escribe la pregunta. 2) El sistema la convierte en una especificación JSON (LLM o intérprete local). 3) La valida contra listas blancas. 4) Ejecuta una consulta parametrizada de solo lectura para su empresa. 5) Muestra tabla y resumen; registra el reporte. *Alternos:* 3a) Especificación inválida → se rechaza.

*Los casos de uso restantes (CU-01…CU-08, CU-11, CU-12, CU-15, CU-17, CU-18, CU-20, CU-24…CU-27) siguen el mismo patrón y se trazan en el Anexo A.*

# CAPÍTULO III. ANÁLISIS Y DISEÑO CON ICONIX

> Todos los diagramas de este capítulo se renderizan desde `docs/uml/src/*.puml` y `docs/uml/generated/*.puml` con `docs/uml/scripts/build_uml.sh`.

## 3.1. Modelo de dominio

| Modelo de dominio |
|---|

*Modelo de dominio*

Conceptos principales: **Empresa (Tenant)** agrupa Sucursales, Usuarios, Clientes, Cajas, Productos, Inventario, Ventas y Reservas. La **Suscripción** vincula la empresa a un **Plan**. El **Cliente** puede tener una cuenta digital (Usuario). Las **Operaciones offline** y sus **Conflictos** sostienen la sincronización. Las entidades globales (ciudades, categorías, tallas, colores, temporadas, colecciones, promociones, proveedores) son compartidas.

## 3.2. Diagramas de robustez

En ICONIX, los diagramas de robustez conectan el **actor** con **fronteras** (pantallas), **controles** (lógica) y **entidades** (datos). Se elaboraron para los casos de uso nuevos o modificados:

| Diagrama | Caso de uso | Archivo |
|---|---|---|
| Venta POS con caja y sincronización | CU-13, CU-14, CU-16 | `03a_robustez_venta_pos.puml` |
| Tienda pública y registro de cliente | CU-21, CU-22 | `03b_robustez_tienda_publica.puml` |
| Reserva y pago | CU-24, CU-25 | `03c_robustez_reserva_pago.puml` |
| Tienda web personalizable | CU-10 | `03d_robustez_tienda_web.puml` |
| Vestidor virtual | CU-23 | `03e_robustez_vestidor.puml` |

**Robustez: venta POS**

*Robustez: venta POS*

**Robustez: tienda pública y registro**

*Robustez: tienda pública y registro*

**Robustez: reserva y pago**

*Robustez: reserva y pago*

**Robustez: tienda web**

*Robustez: tienda web*

**Robustez: vestidor virtual**

*Robustez: vestidor virtual*

## 3.3. Diagramas de secuencia UML

| Diagrama | Descripción |
|---|---|
| `04a_sec_login_multitenant` | Inicio de sesión: el `tenantId` entra al JWT desde el usuario. |
| `04b_sec_registro_cliente_tienda` | Registro de cliente con `X-Tenant-Slug` y `requireTenantId()`. |
| `04c_sec_venta_pos_offline` | Venta offline, cola, lote y sincronización idempotente. |
| `04d_sec_caja` | Crear caja y abrir sesión de caja. |
| `04e_sec_reserva_pago` | Reserva con pago Stripe y webhook. |
| `04f_sec_tienda_web` | Personalización y visita de la tienda web. |
| `04g_sec_reporte_ia` | Reporte en lenguaje natural sin SQL generado por la IA. |

**Secuencia: inicio de sesión**

*Secuencia: inicio de sesión*

**Secuencia: registro de cliente**

*Secuencia: registro de cliente*

**Secuencia: venta offline**

*Secuencia: venta offline*

**Secuencia: cajas**

*Secuencia: cajas*

**Secuencia: reserva y pago**

*Secuencia: reserva y pago*

**Secuencia: tienda web**

*Secuencia: tienda web*

**Secuencia: reporte IA**

*Secuencia: reporte IA*

## 3.4. Diagrama de clases UML

**Modelo persistente (generado).** El script `gen_entities_uml.py` lee las 34 entidades TypeORM y produce el diagrama de clases; así no puede divergir del código.

**Diagrama de clases generado**

*Diagrama de clases generado*

**Clases de diseño.** Muestra el middleware de contexto, subscriber, guards y servicios principales y sus dependencias reales.

**Clases de diseño**

*Clases de diseño*

## 3.5. Diseño de la base de datos

PostgreSQL con el esquema generado por TypeORM. El aislamiento se refuerza en la base de datos con **claves foráneas compuestas `(id, tenantId)`**, de modo que un registro no puede referenciar datos de otra empresa. Entre las tablas nuevas o cambiadas: `cash_registers` (cajas; única por sucursal y nombre), `cash_sessions` (sesión sobre una caja), `audit_logs` (bitácora), `offline_sync_operations` y conflictos, `ai_report_logs`, y la columna `tenants.storeConfig` (jsonb) con la configuración de la tienda web.

**Modelo físico de la base de datos**

*Modelo físico de la base de datos*

El diccionario de datos completo (campos, tipos, PK, nulos) se genera en `docs/uml/generated/diccionario_datos.md` y se incluye en el Anexo B.

### 3.6. Diseño de interfaces de usuario

| Zona | Ruta | Descripción |
|---|---|---|
| Landing SaaS | `/` | Presentación del software, funciones, planes, registro y acceso por código de tienda. |
| Planes y registro de empresa | `/pricing`, `/saas/register` | Elección de plan y alta con verificación por correo. |
| Login | `/login` | Acceso universal; con `?tenant=` muestra la empresa. |
| Tienda de la empresa | `/t/<código>` | Portada personalizada, catálogo, detalle, vestidor, carrito, reservas, checkout. |
| Registro de cliente | `/register?tenant=<código>` | Alta del cliente dentro de la empresa. |
| Cuenta del cliente | `/account/*` | Perfil, reservas, compras. |
| Panel de personal | `/dashboard/*` | Menú lateral por rol: dashboard, ventas, reservas, cajas, productos, inventario, **Mi tienda web**, reportes, IA, analítica, sucursales, usuarios, conflictos, bitácora, configuración. |
| POS | `/dashboard/pos` | Terminal del cajero. |
| App móvil | Expo | Sin sesión: Login, Registro (con código de tienda), Activar cuenta. Con sesión: catálogo, carrito, reservas, compras, perfil, vestidor. |

Criterios de diseño: paleta verde/dorado, tipografía Outfit, tema claro/oscuro configurable, sidebar compacto opcional, diseño responsive y accesible.

## CAPÍTULO IV. DESARROLLO E IMPLEMENTACIÓN

### 4.1. Herramientas y tecnologías utilizadas

| Capa | Tecnología | Uso |
|---|---|---|
| Backend | NestJS 12, TypeScript | API REST modular, guards, |

| Capa | Tecnología | Uso |
|---|---|---|
| | | interceptores |
| Persistencia | TypeORM 1.x, PostgreSQL | Entidades, subscribers, transacciones, bloqueo pesimista |
| Frontend web | React 19, Vite, React Router 7, Axios | Landing, tienda por empresa, panel, POS |
| PWA / offline | vite-plugin-pwa, Workbox, IndexedDB | Navegación y cola offline |
| Vestidor virtual | TensorFlow.js + MoveNet (pose-detection) | Detección de pose en el dispositivo |
| Móvil | Expo ~57, React Native, React Navigation, expo-sqlite | App de clientes |
| Imágenes | Cloudinary | Almacenamiento y optimización |
| Pagos | Stripe (Checkout + webhook) y pago QR simulado | Cobro en línea |
| Notificaciones | Firebase Admin, WebSocket (gateway) | Push y tiempo real |
| Correo | Nodemailer (SMTP) | Códigos de verificación y contraseñas temporales |
| IA / ML | LLM compatible OpenAI (opcional), intérprete local, regresión y K-Means propios | Reportes en lenguaje natural, demanda, RFM |
| Modelado | PlantUML + scripts Python | Modelos UML versionados y verificados |
| Pruebas | Jest, scripts E2E en Node, Playwright (verificación de UI) | Capítulo V |

## 4.2. Diseño de la arquitectura del sistema

| Arquitectura del sistema |
|---|

*Arquitectura del sistema*

**Arquitectura en capas y multitenant.** Cada petición atraviesa:

1. `TenantContextMiddleware`: abre un contexto por petición (AsyncLocalStorage). Si viene `X-Tenant-Slug` resuelve la empresa pública; si viene un JWT válido, el `tenantId` del token manda sobre la cabecera.
2. `AuthGuard` → `TenantGuard` → `RolesGuard` → `PlanFeatureGuard`: autentican, fijan el tenant y validan rol y plan.

3. Servicios de módulo: filtran siempre por `tenantId`. La tienda pública usa `requireTenantId()`, que responde 404 si el código no existe y 400 si no se indicó tienda (se eliminó el antiguo *fallback* a una empresa por defecto).
4. `TenantSubscriber` asigna el `tenantId` en cada inserción y las FKs compuestas de la base de datos impiden referencias cruzadas.

**Enrutamiento del frontend.** `/` es la landing del SaaS; la tienda de cada empresa vive en `/t/:slug/*` (`StoreProvider` carga la empresa y aísla el carrito en `boutique_cart:<slug>`); un interceptor de Axios añade `X-Tenant-Slug` cuando la URL es `/t/<slug>`; el panel está en `/dashboard/*` y la cuenta del cliente en `/account/*`.

**App móvil.** Sin sesión solo hay Login, Registro (con código de tienda) y Activar cuenta; con sesión se monta el catálogo de la empresa del usuario.

## 4.3. Implementación de la base de datos

El esquema se define en 34 entidades TypeORM (diccionario de datos en el Anexo B). Decisiones clave:

- **Claves foráneas compuestas** `(id, tenantId)` declaradas en las entidades para blindar el aislamiento incluso con `synchronize`.
- `cash_registers` (única por `tenantId+branchId+name`) y `cash_sessions` con índices únicos parciales: un cajero con una sola sesión abierta y una caja con una sola sesión abierta.
- `offline_sync_operations` con unicidad `(tenantId, operationId)` para la idempotencia.
- `audit_logs` con usuario, roles, método, ruta, estado HTTP, IP y detalle.
- `tenants.storeConfig` (jsonb) con la configuración de la tienda web, normalizada en servidor.
- Migración idempotente `npm run migrate:multitenant` para datos históricos (RNF-06) y **datos demo** `npm run seed:demo` (2 empresas, contraseña `123456`).

## 4.4. Desarrollo de los módulos del sistema

| Módulo | Backend | Frontend / móvil | Estado |
|---|---|---|---|
| Plataforma SaaS | `tenants` | `LandingPage`, `PricingPage`, `TenantRegisterPage` | Implementado |
| Autenticación y personal | `auth` (`AuthService`, `AccessService`) | `LoginPage`, `RegisterPage`, `ActivateAccountPage`, `UsersPage`; móvil `Login/Register/A` | Implementado |

| Módulo | Backend | Frontend / móvil | Estado |
|---|---|---|---|
| | | `ctivate` | |
| Sucursales | `branches` | `BranchesPage` | Implementado |
| Catálogo | `catalog` + `CloudinaryServic e` | `ProductsAdminPag e` | Implementado |
| Inventario y proveedores | `inventory` | `InventoryAdminPa ge`, `SuppliersPage` | Implementado |
| Cajas y sesiones | `cash-registers`, `cash-sessions` | `CashSessionsAdmi nPage` (crear caja, abrir/cerrar) | Implementado |
| POS | `pos`, `sales` | `PosTerminalPage`, `PosDashboardPage` | Implementado |
| Reservas y pagos | `reservations`, `payment-gateway`, `payments` | `CreateReservatio nPage`, `ReservationsAdmi nPage`, `QrPaymentPage` | Implementado |
| Tienda pública | `store` | `StoreLayout`, `HomePage`, `CatalogPage`, `ProductDetailPag e`, `CartPage`, `CheckoutPage` | Implementado |
| Mi tienda web | `tenants` (`updateStoreConf ig`) | `StoreBuilderPage` , `StoreLinkCard` | Implementado |
| Vestidor virtual | `store` (`getTryOnData`) | `TryOnPage` (recorte y ajuste de la prenda) | Implementado |
| Sincronización offline | `sync` | `offline/db.ts`, `syncManager.ts`, `SyncConflictsPag e` | Implementado |
| Reportes y dashboard | `reports`, `dashboard` | `ReportsPage` (dictado por voz), `DashboardPage` | Implementado |
| IA y ML | `ai-reports`, `analytics`, `intelligence` | `AiReportsPage`, `AnalyticsPage` | Implementado |

| Módulo | Backend | Frontend / móvil | Estado |
|---|---|---|---|
| Bitácora | `audit` (`AuditInterceptor`) | `AuditPage` | Implementado |
| Configuración | — | `SettingsPage` (tema, idioma, moneda) | Implementado |
| Notificaciones | `notifications`, gateway WebSocket, Firebase | Campana de notificaciones | Implementado |

### 4.5. Integración de componentes

- **Web ↔ API:** Axios con interceptores (JWT y `X-Tenant-Slug`); el carrito y la sesión de pago regresan a `/t/<slug>`.
- **Stripe:** `PaymentGatewayService` arma la sesión con el nombre de la empresa y las URL de retorno de su tienda; el webhook confirma el pago.
- **Cloudinary:** las imágenes de producto y de las partes del vestidor se suben mediante el backend.
- **Offline:** la cola local (IndexedDB en web, SQLite en móvil) se envía por lote a `POST /sync/batch`.
- **Móvil ↔ API:** mismo contrato REST; el JWT lleva el `tenantId`, por lo que el catálogo móvil no necesita enviar el código de tienda tras iniciar sesión.
- **Despliegue de referencia:** frontend como sitio estático (PWA), API NestJS en un servicio de aplicaciones, PostgreSQL administrado, y servicios externos (Cloudinary, SMTP, Stripe, Firebase).

## CAPÍTULO V. PRUEBAS Y RESULTADOS

### 5.1. Plan de pruebas

**Objetivo.** Garantizar el aislamiento multitenant, la consistencia transaccional del inventario y de las cajas, la seguridad por rol y por plan, la sincronización offline y el comportamiento de la tienda pública por empresa.

| Nivel | Herramienta | Alcance |
|---|---|---|
| Unitarias | Jest (`npx jest`) | Reglas de sincronización, especificación de reportes IA y SQL, modelos ML |
| Integración / sistema (API + BD reales) | `npm run test:e2e:saas` | Aislamiento, offline, conflictos, IA, ML, planes |
| Integración / sistema (tienda) | `npm run test:e2e:tienda` | Tienda pública, registro por empresa, Mi tienda web |

| Nivel | Herramienta | Alcance |
|---|---|---|
| Interfaz | Playwright (verificación manual asistida) | Landing, tienda por empresa, panel, vestidor |
| Coherencia documental | `python3 docs/uml/scripts/verificar_coherencia.py` | RNF-12: diagramas vs. código |

**Ambientes.** Desarrollo con PostgreSQL local y datos demo (`luna` y `sol`); pasarela en modo de pruebas.

## 5.2. Pruebas funcionales

Se verificó cada módulo por rol con las cuentas demo (todas con contraseña `123456`): administrador, 2 encargados (uno por sucursal), 4 cajeros (2 por sucursal) y un cliente por empresa. Flujos recorridos: alta y plan, personal, catálogo, crear caja → abrir sesión → vender → cerrar, reservas, tienda `/t/luna` y `/t/sol`, registro de cliente, personalización de la tienda, reportes y bitácora.

## 5.3. Pruebas de integración

- **Batería E2E SaaS (`test-saas-e2e.js`): 34 pruebas** (T01–T33) sobre API y PostgreSQL reales — aislamiento entre empresas, POS, sincronización idempotente, conflictos, reportes IA con intento de inyección SQL, analítica ML y restricciones de plan.
- **Batería E2E tienda (`test-tienda-publica.js`): 22 pruebas** (P01–P22) — catálogo por empresa, 404/400 por tienda inexistente o ausente, prioridad del JWT, registro dentro de la empresa, saneamiento y permisos de *Mi tienda web*.

## 5.4. Corrección de errores

Defectos detectados y corregidos durante esta fase:

| # | Defecto | Corrección |
|---|---|---|
| 1 | La tienda pública caía en una empresa por defecto cuando no se indicaba tienda. | `requireTenantId()`: 404/400; se elimina el *fallback*. |
| 2 | El catálogo móvil con sesión devolvía 400 porque las rutas públicas no leían el JWT. | El middleware lee el JWT y su `tenantId` manda sobre la cabecera. |
| 3 | El registro de clientes no se ligaba a una empresa. | `registerClient` exige tienda y asigna `tenantId` a usuario y cliente. |
| 4 | Las URL de retorno de Stripe y el nombre del producto estaban fijos a una marca. | URLs `/t/<slug>` y nombre de la empresa. |
| 5 | Vestidor virtual: prendas chicas | Recorte del espacio transparente, |

| # | Defecto | Corrección |
|---|---|---|
| | y deformadas. | proporción real y escala por cuerpo. |
| 6 | Error al subir imágenes. | Integración y configuración de Cloudinary (variables de entorno). |
| 7 | Cajas: se abría una caja sin haberla creado. | Modelo de cajas: primero se crea, luego se abre una sesión. |
| 8 | Registro fallaba por roles inexistentes en BD vacía. | Siembra de roles al arrancar. |

## 5.5. Resultados de las pruebas

| Prueba | Resultado |
|---|---|
| Jest (unitarias) | **24 / 24 aprobadas** |
| E2E SaaS | **34 / 34 aprobadas** |
| E2E tienda pública | **22 / 22 aprobadas** |
| `tsc` backend, frontend y móvil | Sin errores |
| Coherencia documental | Todos los diagramas coherentes con el código |

*Resultados ejecutados sobre la base de datos de demostración con el código de la rama de entrega. La app móvil se verificó con el compilador de tipos; no se ejecutó en un dispositivo.*

## 5.6. Casos de prueba

| ID | Módulo | Caso de uso | Procedimiento / entrada | Resultado esperado |
|---|---|---|---|---|
| TC-01 | Seguridad | CU-03 | Login de un usuario con contraseña temporal. | JWT con `mustChangePassword=true`; el frontend redirige a `/change-password`. |
| TC-02 | Seguridad | CU-07 | `POST /products` con rol CAJERO. | 403 Forbidden; el catálogo no cambia. |
| TC-03 | Aislamiento | CU-21 | `GET /store/products` con slug | Catálogos distintos (P03). |

| ID | Módulo | Caso de uso | Procedimiento / entrada | Resultado esperado |
|---|---|---|---|---|
| | | | `luna` y con `sol`. | |
| TC-04 | Aislamiento | CU-21 | Slug inexistente / sin slug y sin sesión. | 404 / 400 (P04, P05). |
| TC-05 | Aislamiento | CU-21 | Cliente de luna con cabecera `X-Tenant-Slug: sol`. | Ve el catálogo de luna (P08). |
| TC-06 | Registro | CU-22 | Registro con `X-Tenant-Slug: sol`. | Usuario y cliente creados en sol (P09–P10). |
| TC-07 | Cajas | CU-13 | Abrir sesión con una caja ya abierta. | 409 «Esa caja ya está abierta / ya tienes una caja abierta». |
| TC-08 | Cajas | CU-13 | Abrir sesión en sucursal sin cajas. | Error solicitando crear una caja. |
| TC-09 | POS | CU-14 | Venta sin stock suficiente. | Línea rechazada; stock intacto. |
| TC-10 | Offline | CU-16 | Reenviar la misma operación (mismo UUID). | `DUPLICATE`, sin duplicar la venta (T16). |
| TC-11 | Offline | CU-17 | Vender offline más que el stock real. | Venta aplicada con conflicto `STOCK_SHORTFALL` y stock ≥ 0 (T17, T19). |
| TC-12 | Planes | CU-19 | Plan Básico solicita reporte con IA. | 403 (T30). |
| TC-13 | IA | CU-19 | Pregunta con intento de inyección SQL. | La BD no se altera (T29). |
| TC-14 | Tienda web | CU-10 | Guardar color `rojo` y logo `javascript:…`. | Se usan color por defecto y logo vacío (P16, P18). |
| TC-15 | Tienda web | CU-10 | Cliente intenta `PATCH` | 403 (P21). |

| ID | Módulo | Caso de uso | Procedimiento / entrada | Resultado esperado |
|---|---|---|---|---|
| | | | `/saas/tenants/me/store`. | |
| TC-16 | Móvil | CU-22 | Abrir la app sin sesión. | Solo Login, Registro y Activar cuenta. |
| TC-17 | Vestidor | CU-23 | Prenda con márgenes transparentes y rodillas fuera de cámara. | La prenda se ajusta a caderas con proporción real. |
| TC-18 | Auditoría | CU-11 | Crear un usuario desde el panel. | Registro en bitácora con usuario, IP y ruta. |

*Los casos TC-01 a TC-15 están respaldados por pruebas automáticas o por la verificación de reglas del servidor citadas entre paréntesis. TC-16 (móvil) y TC-17 (vestidor) se verificaron solo por compilación y revisión de código; requieren validación en dispositivo y con cámara. TC-18 se verificó manualmente con la bitácora de datos demo.*

# CAPÍTULO VI. CONCLUSIONES Y RECOMENDACIONES

## 6.1. Conclusiones

1. La plataforma pasó de una tienda única a un **SaaS multitenant** operativo: cada empresa tiene su panel, su tienda en `/t/<código>` y su configuración, y ningún dato cruza de una empresa a otra (pruebas E2E T01–T13 y P01–P12).
2. El aislamiento en tres capas (API, ORM y base de datos) resultó suficiente: incluso un cliente que envía la cabecera de otra empresa recibe solo el catálogo de la suya (P08).
3. La metodología **ICONIX** permitió llegar a la implementación con trazabilidad: requisitos → casos de uso → robustez → secuencia → clases → código → pruebas.
4. El requisito **RNF-12** se cumplió con automatización: los diagramas de clases y de base de datos se generan desde las entidades, y los demás se verifican contra los controladores y servicios reales; un diagrama que cite un endpoint o clase inexistente hace fallar la verificación.
5. La operación offline con sincronización idempotente y gestión de conflictos permite vender en efectivo y reservar sin conexión sin duplicar ni perder dinero (T14–T26).
6. El modelo de **cajas** (crear → abrir sesión → vender → cerrar) y la bitácora dieron control operativo y de auditoría al personal de cada sucursal.
7. El vestidor virtual, la IA en lenguaje natural y la analítica ML aportan diferenciación, con salvaguardas: la IA nunca genera SQL y el procesamiento de pose ocurre en el dispositivo.

## 6.2. Recomendaciones

1. **Correo por empresa.** Hoy un correo pertenece a una sola empresa; para permitir que una persona compre en varias, modelar la pertenencia como relación usuario–empresa.
2. **Catálogos compartidos.** Evaluar volver por empresa las categorías, tallas, colores y colecciones si las empresas necesitan catálogos propios.
3. **Constructor de tienda.** Ampliar *Mi tienda web* con banners, orden de secciones, categorías destacadas y dominio propio.
4. **Migraciones.** Pasar de `synchronize` a migraciones versionadas en producción.
5. **App móvil.** Ejecutar pruebas en dispositivos reales (Android/iOS) y publicar la compilación; permitir ingresar a la tienda con un enlace (deep link) en lugar del código manual.
6. **IA.** Configurar `AI_API_KEY` en producción para lenguaje libre; mantener el intérprete local como respaldo.
7. **Seguridad y operación.** Rotación de secretos, límite de intentos de inicio de sesión, copias de seguridad programadas y monitoreo.
8. **Vestidor.** Subir imágenes de prenda con fondo transparente y accesibles por CORS para el mejor ajuste.

# BIBLIOGRAFÍA


- Chakraborty, S., Hoque, M. S., Jeem, N. R., Biswas, M. C., Bardhan, D., & Lobaton, E. (2021). *Fashion recommendation systems, models and methods: A review*. Informatics, 8(3), 49. https://doi.org/10.3390/informatics8030049
- Chen, C., Ni, J., & Zhang, P. (2024). *Virtual try-on systems in fashion consumption: A systematic review*. Applied Sciences, 14(24), 11839. https://doi.org/10.3390/app142411839
- Shirkhani, S., Mokayed, H., Saini, R., et al. (2023). *Study of AI-driven fashion recommender systems*. SN Computer Science, 4, 514. https://doi.org/10.1007/s42979-023-01932-9
- UNCTAD. (2025). *E-commerce*. SDG Pulse 2025.
- PrestaShop Project. (s. f.). *About the PrestaShop Project*.
- WooCommerce. (s. f.). *About WooCommerce*.
- Rosenberg, D., & Stephens, M. (2007). *Use Case Driven Object Modeling with UML: Theory and Practice*. Apress.
- Rosenberg, D., Stephens, M., & Collins-Cope, M. (2005). *Agile Development with ICONIX Process: People, Process, and Pragmatism*. Apress.
- Object Management Group. (s. f.). *Unified Modeling Language (UML) Specification*.
- NestJS. (s. f.). *NestJS documentation*. https://docs.nestjs.com
- TypeORM. (s. f.). *TypeORM documentation*. https://typeorm.io
- The PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. https://www.postgresql.org/docs/
- PlantUML. (s. f.). *PlantUML Language Reference Guide*. https://plantuml.com
- TensorFlow. (s. f.). *Pose detection (MoveNet)*. https://github.com/tensorflow/tfjs-models/tree/master/pose-detection

- Stripe. (s. f.). *Stripe Checkout and webhooks documentation*. https://docs.stripe.com

# ANEXOS

## Anexo A. Matriz de trazabilidad (requisito → caso de uso → implementación → prueba)

| Requisito | Caso de uso | Implementación principal | Pruebas |
|---|---|---|---|
| RF-01, RF-02 | CU-01, CU-02 | `tenants.service.ts` (`createTenant`, `verifyEmail`, `changePlan`) | T-SaaS, T10, T30, T33 |
| RF-03 | CU-03 | `auth.service.ts` (`login`, `activateAccount`, `changePassword`) | TC-01, P06 |
| RF-04 | CU-05 | `branches.service.ts` (límite `maxBranches`) | T10 |
| RF-05 | CU-06 | `access.service.ts`, `UsersPage.tsx` | TC-18 |
| RF-06, RF-07 | CU-07 | `catalog.service.ts`, `cloudinary.service.ts` | TC-02 |
| RF-08 | CU-08 | `inventory.service.ts` | T-SaaS (stock) |
| RF-09 | CU-09, CU-13 | `cash-registers.service.ts`, `cash-sessions.service.ts` | TC-07, TC-08 |
| RF-10 | CU-14 | `pos.service.ts`, `PosTerminalPage.tsx` | T14, TC-09 |
| RF-11, RF-12 | CU-16, CU-17 | `sync.service.ts`, `sync.rules.ts`, `offline/*` | T14–T26, Jest `sync.rules.spec` |
| RF-13 | CU-21 | `store.service.ts` | P01–P08 |

| Requisito | Caso de uso | Implementación principal | Pruebas |
|---|---|---|---|
| | | (`requireTenantId`), `StoreContext.tsx` | |
| RF-14 | CU-22 | `auth.service.ts` (`registerClient`), `RegisterPage.tsx`, `RegisterScreen.tsx`, `AppNavigator.tsx` | P09–P12, TC-16 |
| RF-15 | CU-15, CU-24 | `reservations.service.ts`, `payment-gateway.service.ts` | T21, T22 |
| RF-16 | CU-25, CU-26 | `sales.service.ts` (`checkoutWeb`) | — |
| RF-17 | CU-23 | `TryOnPage.tsx`, `utils/garmentFit.ts` | TC-17 |
| RF-18 | CU-10 | `tenants.service.ts` (`updateStoreConfig`), `StoreBuilderPage.tsx` | P13–P22 |
| RF-19 | CU-18 | `dashboard.service.ts`, `reports.service.ts` | — |
| RF-20 | CU-19 | `ai-reports.service.ts`, `report-spec.ts` | T27–T30, Jest |
| RF-21 | CU-20 | `analytics.service.ts`, `ml.ts` | T31–T33, Jest `ml.spec` |
| RF-22 | CU-11 | `audit.interceptor.ts`, `audit.service.ts` | TC-18 |

| Requisito | Caso de uso | Implementación principal | Pruebas |
|---|---|---|---|
| RNF-01, RNF-02 | CU-03, CU-21 | `common/ tenancy/*`, FKs compuestas | T01–T13, P03–P12 |
| RNF-12 | — | `docs/uml/ scripts/*` | `verificar_cohere ncia.py` |

## Anexo B. Diccionario de datos

El diccionario completo se genera automáticamente desde las entidades en `docs/uml/generated/diccionario_datos.md` (campos, tipos, claves primarias y nulabilidad de las 34 tablas). Se incluye a continuación.

### *Diccionario de datos (generado)*

_Generado por docs/uml/scripts/gen_entities_uml.py a partir de backend/src/**/*.entity.ts_

#### ai_report_logs (`AiReportLog`, módulo ai-reports)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| userId | `number` | | |
| question | `string` | | |
| spec | `any` | | sí |
| engine | `'LLM' \| 'REGLAS'` | | |
| rowCount | `number` | | |
| error | `string \| null` | | sí |
| createdAt | `Date` | | |

#### audit_logs (`AuditLog`, módulo audit)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| userId | `number \| null` | | sí |
| userName | `string \| null` | | sí |
| userEmail | `string \| null` | | sí |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| userRoles | `string \| null` | | sí |
| branchId | `number \| null` | | sí |
| action | `string` | | |
| entity | `string \| null` | | sí |
| entityId | `string \| null` | | sí |
| summary | `string` | | |
| details | `string \| null` | | sí |
| method | `string` | | |
| path | `string` | | |
| statusCode | `number` | | |
| ip | `string \| null` | | sí |
| userAgent | `string \| null` | | sí |
| createdAt | `Date` | | |

### roles (`Role`, módulo auth)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| description | `string` | | sí |
| active | `boolean` | | |

### user_roles (`UserRole`, módulo auth)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| userId | `number` | | |
| roleId | `number` | | |
| active | `boolean` | | |
| assignedAt | `Date` | | |

### users (`User`, módulo auth)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| name | `string` | | |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| email | `string` | | |
| passwordHash | `string` | | |
| active | `boolean` | | |
| mustChangePassword | `boolean` | | |
| temporaryPasswordExpiresAt | `Date \| null` | | sí |
| emailVerified | `boolean` | | |
| verificationCodeHash | `string \| null` | | sí |
| verificationCodeExpiresAt | `Date \| null` | | sí |
| verificationAttempts | `number` | | |
| createdAt | `Date` | | |
| branchId | `number` | | sí |

### branches (`Branch`, módulo branches)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| name | `string` | | |
| address | `string` | | |
| phone | `string` | | |
| active | `boolean` | | |
| createdAt | `Date` | | |
| cityId | `number` | | |

### cities (`City`, módulo branches)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| active | `boolean` | | |

### categories (`Category`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| active | `boolean` | | |

### collections (`Collection`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| description | `string` | | sí |
| seasonId | `number` | | |
| active | `boolean` | | |

### colors (`Color`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| hexCode | `string` | | sí |
| active | `boolean` | | |

### products (`Product`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| name | `string` | | |
| price | `number` | | |
| active | `boolean` | | |
| categoryId | `number` | | |
| imageUrl | `string` | | sí |
| imagePublicId | `string` | | sí |
| colorizable | `boolean` | | |
| sourceColor | `string` | | sí |
| arEnabled | `boolean` | | |
| arImageUrl | `string` | | sí |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| arImagePublicId | `string` | | sí |
| arTorsoUrl | `string` | | sí |
| arTorsoPublicId | `string` | | sí |
| arLeftSleeveUrl | `string` | | sí |
| arLeftSleevePublicId | `string` | | sí |
| arRightSleeveUrl | `string` | | sí |
| arRightSleevePublicId | `string` | | sí |
| arType | `ArGarmentType` | | sí |
| collectionId | `number` | | sí |

### promotions (`Promotion`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| type | `'PERCENTAGE' \| 'FIXED'` | | |
| value | `number` | | |
| startDate | `string` | | |
| endDate | `string` | | |
| active | `boolean` | | |

### seasons (`Season`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| startDate | `string` | | |
| endDate | `string` | | |
| active | `boolean` | | |

### sizes (`Size`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| active | `boolean` | | |

### variants (`Variant`, módulo catalog)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| sku | `string` | | |
| active | `boolean` | | |
| productId | `number` | | |
| sizeId | `number` | | |
| colorId | `number` | | |

### clients (`Client`, módulo clients)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| name | `string` | | |
| lastName | `string` | | |
| email | `string` | | |
| phone | `string` | | sí |
| registrationDate | `Date` | | |
| active | `boolean` | | |
| userId | `number` | | sí |
| activationCodeHash | `string \| null` | | sí |
| activationCodeExpiresAt | `Date \| null` | | sí |

### inventory (`Inventory`, módulo inventory)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| stock | `number` | | |
| reserved | `number` | | |
| stockMin | `number` | | |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| stockMax | `number` | | sí |
| updatedAt | `Date` | | |
| branchId | `number` | | |
| variantId | `number` | | |

### inventory_movements (`InventoryMovement`, módulo inventory)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| quantity | `number` | | |
| movementDate | `Date` | | |
| type | `MovementType` | | |
| observation | `string` | | sí |
| inventoryId | `number` | | |
| userId | `number` | | sí |

### suppliers (`Supplier`, módulo inventory)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| name | `string` | | |
| nit | `string` | | |
| email | `string` | | sí |
| phone | `string` | | sí |
| address | `string` | | sí |
| active | `boolean` | | |

### notification_devices (`NotificationDevice`, módulo notifications)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| userId | `number` | | |
| token | `string` | | |
| platform | `string` | | |
| active | `boolean` | | |

### notifications (`Notification`, módulo notifications)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| userId | `number` | | |
| type | `string` | | |
| title | `string` | | |
| message | `string` | | |
| reservationId | `number` | | sí |
| read | `boolean` | | |
| createdAt | `Date` | | |

### reservation_items (`ReservationItem`, módulo reservations)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| quantity | `number` | | |
| reservationId | `number` | | |
| variantId | `number` | | |

### reservations (`Reservation`, módulo reservations)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| offlineOperationId | `string \| null` | | sí |
| status | `ReservationStatus` | | |
| date | `string` | | |
| approximateTime | `string` | | sí |
| registrationDate | `Date` | | |
| clientId | `number` | | |
| branchId | `number` | | |

### cash_registers (`CashRegister`, módulo sales)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| branchId | `number` | | |
| name | `string` | | |
| active | `boolean` | | |
| createdAt | `Date` | | |

### cash_sessions (`CashSession`, módulo sales)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| branchId | `number` | | |
| cashierId | `number` | | |
| registerId | `number \| null` | | sí |
| openingAmount | `number` | | |
| openedAt | `Date` | | |
| closingAmount | `number` | | sí |
| expectedAmount | `number` | | sí |
| difference | `number` | | sí |
| closedAt | `Date` | | sí |
| openedById | `number \| null` | | sí |
| closedById | `number \| null` | | sí |
| status | `CashSessionStatus` | | |

### payments (`Payment`, módulo sales)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| amount | `number` | | |
| method | `PaymentMethod` | | |
| status | `PaymentStatus` | | |
| date | `Date` | | |
| transactionReference | `string` | | sí |
| stripePaymentInten | `string` | | sí |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| tId | | | |
| externalReference | `string` | | sí |
| provider | `string` | | sí |
| expiresAt | `Date` | | sí |
| saleId | `number` | | |

### sale_items (`SaleItem`, módulo sales)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| quantity | `number` | | |
| unitPrice | `number` | | |
| subtotal | `number` | | |
| saleId | `number` | | |
| variantId | `number` | | |

### sales (`Sale`, módulo sales)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| offlineOperationId | `string \| null` | | sí |
| channel | `SaleChannel` | | |
| status | `SaleStatus` | | |
| date | `Date` | | |
| subtotal | `number` | | |
| discount | `number` | | sí |
| total | `number` | | |
| branchId | `number` | | |
| clientId | `number` | | sí |
| userId | `number` | | sí |
| cashSessionId | `number` | | sí |
| reservationId | `number` | | sí |

### offline_sync_operations (`SyncOperation`, módulo sync)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| operationId | `string` | | |
| type | `SyncOperationType` | | |
| status | `SyncOperationStatus` | | |
| deviceId | `string \| null` | | sí |
| userId | `number` | | |
| payload | `any` | | |
| result | `any` | | sí |
| clientCreatedAt | `Date` | | |
| receivedAt | `Date` | | |

### sync_conflicts (`SyncConflict`, módulo sync)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| operationId | `string` | | |
| type | `SyncConflictType` | | |
| status | `SyncConflictStatus` | | |
| branchId | `number` | | sí |
| saleId | `number` | | sí |
| message | `string` | | |
| detail | `any` | | sí |
| resolutionAction | `string \| null` | | sí |
| resolutionNotes | `string \| null` | | sí |
| resolvedBy | `number` | | sí |
| resolvedAt | `Date \| null` | | sí |
| createdAt | `Date` | | |

### plans (`Plan`, módulo tenants)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| code | `string` | | sí |
| name | `string` | | |
| priceMonthly | `number` | | |
| maxBranches | `number` | | |
| aiGenerativeReports | `boolean` | | |
| offlineOperations | `boolean` | | |
| createdAt | `Date` | | |

### subscription_history (`SubscriptionHistory`, módulo tenants)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| planId | `number` | | |
| action | `string` | | |
| previousPlanId | `number` | | sí |
| changedAt | `Date` | | |
| notes | `string` | | sí |

### subscriptions (`Subscription`, módulo tenants)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |
| tenantId | `number` | | |
| planId | `number` | | |
| status | `string` | | |
| startDate | `Date` | | |
| endDate | `Date` | | sí |
| updatedAt | `Date` | | |

### tenants (`Tenant`, módulo tenants)

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| id | `number` | sí | |

| Campo | Tipo | PK | Nulo |
|---|---|---|---|
| slug | `string` | | |
| name | `string` | | |
| taxId | `string` | | sí |
| active | `boolean` | | |
| storeConfig | `json \| null` | | sí |
| createdAt | `Date` | | |
| updatedAt | `Date` | | |

# Anexo C. Guía de usuario

## Cuentas de demostración (contraseña 123456)

Se crean con `npm run seed:demo` (dos empresas: `luna` y `sol`). Los correos siguen el patrón `admin@<empresa>.com`, `encargado1@<empresa>.com`, `cajero1@<empresa>.com`, `cliente@<empresa>.com`.

## Cliente

1. Abra el enlace que le compartió su tienda (`/t/<código>`) o escriba el código en la página principal.
2. Explore el catálogo, elija talla y color y vea la disponibilidad por sucursal.
3. Cree su cuenta en **Registrarse** (queda asociada a esa tienda) e inicie sesión.
4. **Vestidor virtual:** en el detalle de la prenda pulse *Probar*, autorice la cámara y párese de modo que se vea del torso a las rodillas.
5. Reserve eligiendo sucursal, fecha y hora, y pague en línea o en la sucursal. Consulte **Mi cuenta** para reservas y compras.
6. **App móvil:** inicie sesión con su correo (la primera vez, regístrese con el código de tienda); verá solo el catálogo de su empresa.

## Cajero

1. Inicie sesión; su inicio es el **Terminal POS**.
2. Abra su caja (la sucursal debe tener cajas creadas) indicando el monto inicial.
3. Agregue prendas, elija el método de pago y cobre. Sin conexión solo se acepta efectivo; las ventas se sincronizan al volver la red.
4. Entregue reservas desde **Reservas** y cierre su caja al terminar el turno.

## Encargado de sucursal

Mismo panel que el administrador pero limitado a su sucursal: crear cajas, inventario, ventas, reservas, resolver conflictos de sincronización y reportes de su sucursal.

### Administrador

1. **Sucursales** y **Usuarios**: cree sucursales (según el plan) y asigne encargados y cajeros.
2. **Productos** e **Inventario**: cargue productos, variantes e imágenes y registre el stock por sucursal.
3. **Mi tienda web**: personalice portada, logo, color y contacto; copie o comparta el enlace; revise la vista previa.
4. **Reportes**, **Reportes con IA** y **Analítica ML** (según el plan); **Bitácora** para auditar acciones.
5. **Configuración**: tema, idioma, moneda y barra lateral compacta.

## Anexo D. Cómo reproducir

```
# Modelado UML (genera clases/BD desde el código, verifica coherencia y renderiza)
bash docs/uml/scripts/build_uml.sh              # requiere PlantUML para renderizar (PLANTUML_JAR=...)

# Informe completo en Word y PDF
bash docs/informe/build_informe.sh

# Backend y pruebas
cd backend && npm install && npm run seed:demo && npm run build && npm run start:prod
npx jest                          # 24 pruebas unitarias
npm run test:e2e:saas             # 34 pruebas E2E
npm run test:e2e:tienda           # 22 pruebas E2E
```